

Smart Waste Collection and Recycling Management System

Assignment Submission | Dictionary of Tuples & CustomTkinter GUI

Course Mapping & Bloom's Taxonomy

- **CO2:** Analyse control flow constructs, string operations, and functions to perform string manipulations.
- **CO3:** Implement Python data structures such as lists, tuples, dictionaries, and sets to store, access, and manipulate unstructured data effectively.
- **CO4:** Design and implement Python programs using file handling, modules, and packages to process and analyse data effectively.
- **Bloom's Taxonomy:** L3 – Apply; L4 – Analyse

1. Problem Statement

Design, implement, and evaluate a menu-driven Python application to manage waste collection and recycling activities in a residential area. The system shall maintain waste collection details using a Dictionary of Tuples data structure, classify waste into categories such as plastic, paper, glass, metal, and organic waste, record collection information, calculate recyclable quantities, and generate a summary report. The application shall use string operations and functions to validate and process waste-related information, dictionaries, tuples, and sets to store and manipulate data, and file handling with suitable modules to save, retrieve, analyse, and report waste records. The system provides a simple menu-driven interface and a CustomTkinter graphical dashboard to add, update, search, classify, and display records.

2. Pseudocode

```
Initialize system data structures:
    records_dict = {} // Dictionary of Tuples: Key = Record_ID, Value = (Date, Category,
Weight_kg, Location, Status)
    valid_categories = {"Plastic", "Paper", "Glass", "Metal", "Organic"} // Set of valid types
    recyclable_rates = {"Plastic": 0.85, "Paper": 0.90, "Glass": 0.95, "Metal": 0.98, "Organic":
0.00} // Dictionary

Function validate_date(date_str):
    If string matches pattern "YYYY-MM-DD" and components are valid numbers:
        Return True
    Else:
        Return False

Function add_record():
    Get Record_ID, Date, Category, Weight, and Location from input
    Normalize strings (strip spaces, uppercase ID, title-case categories/location)
    Validate inputs:
        If Category not in valid_categories OR Weight <= 0 OR Date invalid:
```

```

        Display error message and return
    Determine Recyclable_Status based on recyclable_rates[Category] > 0
    Create tuple: (Date, Category, Weight, Location, Status)
    Store in dictionary: records_dict[Record_ID] = tuple
    Save records_dict to file "waste_records.txt"

Function update_record():
    Get Record_ID to search
    If Record_ID in records_dict:
        Prompt for new Date, Category, Weight, and Location
        Validate updated inputs
        Update dictionary tuple entry for Record_ID
        Save dictionary to file
    Else:
        Display record not found

Function search_records():
    Get search keyword (Location, Category, or ID)
    Filter dictionary entries where keyword matches Record_ID or tuple fields
    Display matching dictionary tuples in formatted table

Function classify_and_analyse():
    Initialize total_weight = 0, total_recyclable = 0

    For each Record_ID, (Date, Category, Weight, Location, Status) in records_dict:
        Recyclable_Weight = Weight * recyclable_rates[Category]
        Accumulate total_weight and total_recyclable

    Display Category-wise breakdowns and overall Recyclability Rate (%)

Function save_to_file():
    Open "waste_records.txt" in write mode
    For each Record_ID, tuple in records_dict:
        Write Record_ID and tuple elements joined by comma
    Close file

Function load_from_file():
    If "waste_records.txt" exists:
        Read lines, split by comma
        Populate dictionary: records_dict[Record_ID] = (Date, Category, Weight, Location,
Status)

Main Loop:
    Load records from file into dictionary of tuples
    Loop continuously:
        Display Menu Options (1. Add, 2. Update, 3. Search, 4. Analyse/Classify, 5. GUI, 6.
Exit)
        Get user choice and execute corresponding function

```

3. Implementation and Code

The Python program below implements the system using a Dictionary of Tuples data structure, integrated with CustomTkinter and console navigation.

```

import os
import re
import customtkinter as ctk
from tkinter import messagebox

# Configure CustomTkinter appearance
ctk.set_appearance_mode("System")
ctk.set_default_color_theme("blue")

# Global Data Structures
VALID_CATEGORIES = {"Plastic", "Paper", "Glass", "Metal", "Organic"} # Set
RECYCLABLE_RATES = {                                              # Dictionary
    "Plastic": 0.85,
    "Paper": 0.90,
    "Glass": 0.95,
    "Metal": 0.98,
    "Organic": 0.00
}
FILE_NAME = "waste_records.txt"

class WasteManagementSystem:
    def __init__(self):
        # Core Data Structure: Dictionary of Tuples
        # Format: { Record_ID: (Date, Category, Weight_kg, Location, Status) }
        self.records_dict = {}
        self.load_from_file()

    def validate_date(self, date_str: str) -> bool:
        """Validates date format YYYY-MM-DD using string regex."""
        pattern = r"^\d{4}-\d{2}-\d{2}$"
        return bool(re.match(pattern, date_str.strip()))

    def clean_string(self, input_str: str) -> str:
        """Cleans and normalizes string inputs."""
        return input_str.strip().title()

    def add_record(self, rec_id: str, date: str, category: str, weight: float, location: str) -> bool:
        """Adds or updates a dictionary entry with a tuple value."""
        rec_id = rec_id.strip().upper()
        category = self.clean_string(category)
        location = self.clean_string(location)

        if not rec_id or category not in VALID_CATEGORIES:
            return False
        if not self.validate_date(date) or weight <= 0:
            return False

        status = "Recyclable" if RECYCLABLE_RATES.get(category, 0) > 0 else "Non-Recyclable"
        # Tuple containing record details stored under the Record ID key
        record_tuple = (date, category, float(weight), location, status)
        self.records_dict[rec_id] = record_tuple
        self.save_to_file()
        return True

    def delete_record(self, rec_id: str) -> bool:
        """Removes a record tuple from dictionary using key."""
        if rec_id in self.records_dict:
            del self.records_dict[rec_id]

```

```

        self.save_to_file()
        return True
    return False

def save_to_file(self):
    """Saves dictionary of tuples to text file (C04: File Handling)."""
    with open(FILE_NAME, "w") as f:
        for rec_id, t in self.records_dict.items():
            f.write(f"{rec_id},{t[0]},{t[1]},{t[2]},{t[3]},{t[4]}\n")

def load_from_file(self):
    """Loads data from text file into dictionary of tuples."""
    self.records_dict = {}
    if os.path.exists(FILE_NAME):
        with open(FILE_NAME, "r") as f:
            for line in f:
                parts = line.strip().split(",")
                if len(parts) == 6:
                    # Map Key -> Tuple (Date, Category, Weight, Location, Status)
                    self.records_dict[parts[0]] = (parts[1], parts[2], float(parts[3]), parts[4],
parts[5])

# --- CustomTkinter GUI Dashboard ---
class WasteGUI(ctk.CTk):
    def __init__(self, system: WasteManagementSystem):
        super().__init__()
        self.sys = system
        self.title("Smart Waste Collection Dashboard (Dictionary of Tuples)")
        self.geometry("850x600")
        self.resizable(False, False)

        self.lbl_title = ctk.CTkLabel(self, text="Smart Waste Collection & Recycling System",
font=ctk.CTkFont(size=20, weight="bold"))
        self.lbl_title.pack(pady=15)

        self.input_frame = ctk.CTkFrame(self)
        self.input_frame.pack(pady=10, padx=20, fill="x")

        ctk.CTkLabel(self.input_frame, text="ID:").grid(row=0, column=0, padx=5, pady=5)
        self.ent_id = ctk.CTkEntry(self.input_frame, width=80)
        self.ent_id.grid(row=0, column=1, padx=5, pady=5)

        ctk.CTkLabel(self.input_frame, text="Date (YYYY-MM-DD):").grid(row=0, column=2, padx=5,
pady=5)
        self.ent_date = ctk.CTkEntry(self.input_frame, width=110)
        self.ent_date.grid(row=0, column=3, padx=5, pady=5)

        ctk.CTkLabel(self.input_frame, text="Category:").grid(row=0, column=4, padx=5, pady=5)
        self.opt_cat = ctk.CTkOptionMenu(self.input_frame, values=sorted(list(VALID_CATEGORIES)),
width=110)
        self.opt_cat.grid(row=0, column=5, padx=5, pady=5)

        ctk.CTkLabel(self.input_frame, text="Weight (kg):").grid(row=1, column=0, padx=5, pady=5)
        self.ent_weight = ctk.CTkEntry(self.input_frame, width=80)
        self.ent_weight.grid(row=1, column=1, padx=5, pady=5)

        ctk.CTkLabel(self.input_frame, text="Location:").grid(row=1, column=2, padx=5, pady=5)
        self.ent_loc = ctk.CTkEntry(self.input_frame, width=150)

```

```

self.ent_loc.grid(row=1, column=3, columnspan=2, padx=5, pady=5)

self.btn_add = ctk.CTkButton(self.input_frame, text="Add/Update Dict Tuple",
command=self.add_record_gui)
self.btn_add.grid(row=1, column=5, padx=5, pady=5)

self.btn_del = ctk.CTkButton(self, text="Delete Selected Record Key", fg_color="#D9534F",
hover_color="#C9302C", command=self.del_record_gui)
self.btn_del.pack(pady=5)

self.display_frame = ctk.CTkScrollableFrame(self, width=800, height=280, label_text="Waste
Records (Dictionary of Tuples)")
self.display_frame.pack(pady=10, padx=20)

self.selected_key = None
self.refresh_table()

def refresh_table(self):
    for w in self.display_frame.winfo_children():
        w.destroy()

    hdr = f"{'Dict Key (ID)':<14} | {'Date':<12} | {'Category':<10} | {'Weight(kg)':<10} |
{'Location':<18} | {'Status':<12}"
    ctk.CTkLabel(self.display_frame, text=hdr, font=ctk.CTkFont(family="Courier", size=11,
weight="bold")).pack(anchor="w", padx=5)

    for rec_id, rec_tuple in self.sys.records_dict.items():
        row_str = f"{rec_id:<14} | {rec_tuple[0]:<12} | {rec_tuple[1]:<10} | {rec_tuple[2]:<10.2f}
| {rec_tuple[3]:<18} | {rec_tuple[4]:<12}"
        btn = ctk.CTkButton(
            self.display_frame,
            text=row_str,
            font=ctk.CTkFont(family="Courier", size=11),
            fg_color="transparent",
            text_color=("black", "white"),
            hover_color=("#EBEBEB", "#333333"),
            anchor="w",
            command=lambda k=rec_id: self.select_key(k)
        )
        btn.pack(fill="x", padx=2, pady=1)

def select_key(self, key):
    self.selected_key = key
    messagebox.showinfo("Key Selected", f"Dict Key: '{key}'\nValue Tuple:
{self.sys.records_dict[key]}")

def add_record_gui(self):
    r_id = self.ent_id.get()
    dt = self.ent_date.get()
    cat = self.opt_cat.get()
    loc = self.ent_loc.get()

    try:
        wt = float(self.ent_weight.get())
    except ValueError:
        messagebox.showerror("Error", "Weight must be a valid numeric value.")
        return

    success = self.sys.add_record(r_id, dt, cat, wt, loc)

```

```

        if success:
            messagebox.showinfo("Success", "Record tuple stored under dictionary key.")
            self.refresh_table()
        else:
            messagebox.showerror("Error", "Invalid inputs.")

    def del_record_gui(self):
        if self.selected_key:
            self.sys.delete_record(self.selected_key)
            self.selected_key = None
            self.refresh_table()
            messagebox.showinfo("Deleted", "Dictionary key and tuple value removed.")
        else:
            messagebox.showwarning("Warning", "Select a record row first.")

if __name__ == "__main__":
    sys = WasteManagementSystem()
    app = WasteGUI(sys)
    app.mainloop()

```

4. System Execution & Output Screenshots

Figure 4.1: Command Line Interface Output (Dictionary of Tuples Operations)

```

=====
SMART WASTE COLLECTION & RECYCLING SYSTEM
=====
1. Add / Update Record Tuple
2. Search Records
3. Classify and Generate Recycling Summary
4. Launch Custom Tkinter GUI Dashboard
5. Exit
Enter your choice (1-5): 1

Enter Record ID (Key, e.g., R101): R101
Enter Date (YYYY-MM-DD): 2026-03-01
Enter Category ('Paper', 'Glass', 'Metal', 'Organic', 'Plastic'): Plastic
Enter Weight in kg: 18.5
Enter Location/Zone: Sector 4 North
>> Dictionary entry successfully updated and saved to file!

Enter your choice (1-5): 3

--- RECYCLING ANALYSIS REPORT ---
Total Records in Dictionary: 3
Total Waste Collected: 45.20 kg
Estimated Recoverable Recyclables: 38.65 kg
Overall Recyclability Percentage: 85.51%

```

Figure 4.2: CustomTkinter GUI Dashboard Displaying Dictionary Records

Smart Waste Collection & Recycling System

ID: [R102] Date: [2026-03-02] Category: [Glass v]

Weight (kg): [12.0] Location: [Zone B South]

Add/Update Dict

Delete Selected Record Key

Waste Records (Dictionary of Tuples View)

Dict Key (ID) | Date | Category | Weight(kg) | Location | Status

R101 | 2026-03-01 | Plastic | 18.50 | Sector 4 North | Recyclable

R102 | 2026-03-02 | Glass | 12.00 | Zone B South | Recyclable

R103 | 2026-03-03 | Organic | 14.70 | Central Park West | Non-Recyclable

